

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Projeto Prático ADS 2026/1

Sistema Helpdesk de TI

Sistema Web Fullstack para Gestão de Chamados de Suporte Técnico

1. Identificação

Campo	Descrição
Disciplina	Tópicos Especiais — Análise e Desenvolvimento de Sistemas
Professor	MSc. Alex Junior Nunes
Tema	Helpdesk de TI (sistema web fullstack de gestão de chamados de suporte técnico)
Entrega	Entrega Final (N2) — Aplicação executável e documentação consolidada
Data da entrega / defesa	01/07/2026
Equipe	Felipe Trindade, Higor Bueno, Fernando Vinicius e Nicolas Toshio
Repositório	https://github.com/Trindadefelipe/HelpDeskTi_TFinal
Arquitetura	MVC — Node.js + Express + EJS + PostgreSQL + Bootstrap 5
Versão	V2 — 22/06/2026 (consolidação para a Entrega Final)

Nota de fidelidade: este documento descreve o sistema conforme efetivamente implementado e versionado no repositório oficial. Os módulos marcados como **Em desenvolvimento** estão em andamento e ainda não foram integrados à branch principal (`main`) no momento desta consolidação; sua finalização ocorre na reta final até a defesa (01/07/2026).

2. Apresentação do Sistema e Objetivos

2.1 Apresentação

O **Sistema Helpdesk de TI** é uma aplicação web fullstack voltada para departamentos de TI de pequenas e médias organizações que precisam organizar de forma centralizada a abertura, o acompanhamento e a resolução de chamados de suporte técnico. O sistema substitui o controle informal feito em e-mails, mensagens avulsas e planilhas por um cadastro estruturado de chamados, com atribuição a técnicos responsáveis, classificação por categoria e departamento de origem e histórico completo de movimentações para cada solicitação.

A solução foi construída em arquitetura **MVC**, com autenticação de usuários, controle de acesso por perfil (administrador e usuário comum) e rastreabilidade das movimentações de cada chamado por meio de **histórico automático**.

2.2 Objetivo geral

Desenvolver uma plataforma web fullstack, em arquitetura MVC, que permita ao departamento de TI gerenciar chamados, técnicos, categorias e departamentos, com autenticação de usuários e controle de acesso por perfil, oferecendo rastreabilidade completa das movimentações de cada chamado por meio de histórico automático.

2.3 Objetivos específicos

- Permitir a abertura, edição, listagem e cancelamento (inativação lógica) de chamados, com classificação por categoria, departamento, prioridade e técnico responsável.
- Manter cadastros de apoio para departamentos, técnicos e categorias.
- Registrar automaticamente no histórico do chamado toda mudança de status, compondo uma linha do tempo cronológica das movimentações.
- Preservar o histórico de atendimentos mesmo no cancelamento de um chamado (exclusão lógica / *soft delete*).
- Disponibilizar um painel inicial com indicadores básicos e a listagem dos chamados.
- Garantir que apenas usuários autenticados acessem as rotas internas, com distinção de acesso entre perfil administrador e usuário comum.

3. Requisitos do Sistema (síntese da N1)

Resumo dos requisitos definidos na Entrega 1, agrupados por módulo. A coluna **Situação** indica o estado de implementação na Entrega Final.

3.1 Requisitos Funcionais (RF)

Módulo	RF (N1)	Descrição resumida	Prioridade	Situação
Autenticação	RF01–RF08	Login por e-mail/senha; validação contra hash sem revelar o campo incorreto; mensagens de erro amigáveis; manutenção de sessão; logout; proteção de rotas por middleware; controle de acesso por perfil (admin × comum); bloqueio de usuário inativo.	Alta/Média	Implementado
Departamentos	RF09–RF11	Cadastrar, editar, listar e gerenciar departamentos; vínculo de departamento ao chamado.	Alta/Média	Implementado
Chamados	RF12–RF18	Abertura com título, descrição, categoria e prioridade obrigatórios; data de abertura preenchida automaticamente, com solicitante e departamento informados no formulário de abertura; atribuição a técnico e mudança de status; registro automático de histórico ; exibição do histórico no detalhe.	Alta/Média	Implementado (exceto filtros — RF18, previsto como melhoria futura)
Técnicos	RF19	Cadastrar, editar e listar técnicos, com departamento de atuação.	Alta	Em desenvolvimento
Categorias	RF20	Cadastrar, editar e listar categorias (Hardware, Software, Rede, Acesso).	Alta	Em desenvolvimento
Status	RF21	Manter o ciclo de vida dos chamados: Aberto, Em Atendimento, Resolvido, Cancelado.	Média	Implementado (lista controlada via seed)
Dashboard	RF22	Tela inicial com indicadores (contagem por status) e visão das movimentações.	Baixa	Em desenvolvimento

3.2 Requisitos Não Funcionais (RNF)

RNF (N1)	Descrição	Situação
RNF01	Node.js + Express, com EJS e Bootstrap 5 no front-end.	Atendido
RNF02	Padrão MVC, com pastas separadas para models, views, controllers, routes e middlewares.	Atendido
RNF03	Banco PostgreSQL relacional, com chaves estrangeiras e integridade referencial.	Atendido
RNF04	Senhas armazenadas com hash (bcrypt), nunca em texto plano.	Atendido
RNF05	Rotas internas protegidas por middleware de autenticação.	Atendido
RNF06	Credenciais e string de conexão em variáveis de ambiente (.env), fora do versionamento.	Atendido (.env.example no repositório)
RNF07	Páginas responsivas, navegáveis em desktop, com Bootstrap 5.	Atendido
RNF08	Código versionado no GitHub, com commits por integrante e <i>feature branches</i> .	Atendido
RNF09	Erros de banco e validação tratados sem expor <i>stack trace</i> ao usuário.	Atendido
RNF10	<i>Seed</i> inicial para popular dados de demonstração.	Atendido (seed.sql)

4. Modelo de Dados e Relacionamentos (DER)

O modelo é composto por **sete tabelas** em banco relacional PostgreSQL, com integridade referencial por chave estrangeira, descrito a seguir em termos de entidades, atributos e relacionamentos.

4.1 Entidades e atributos

usuarios — usuários autenticáveis do sistema.

Atributo	Tipo	Restrições
id_usuario (PK)	SERIAL	NOT NULL
nome	VARCHAR	NOT NULL
email	VARCHAR	UNIQUE, NOT NULL
senha_hash	VARCHAR	NOT NULL (hash bcrypt)
perfil	VARCHAR/ENUM	NOT NULL (admin / comum)
ativo	BOOLEAN	DEFAULT TRUE

departamentos — departamentos da organização.

Atributo	Tipo	Restrições
id_departamento (PK)	SERIAL	NOT NULL
nome	VARCHAR	NOT NULL
responsavel	VARCHAR	NULL
localizacao	VARCHAR	NULL

tecnicos — técnicos responsáveis pelo atendimento.

Atributo	Tipo	Restrições
id_tecnico (PK)	SERIAL	NOT NULL
nome	VARCHAR	NOT NULL
telefone	VARCHAR	NULL
email	VARCHAR	NOT NULL
id_departamento (FK)	INT	→ departamentos.id_departamento

status — lista controlada do ciclo de vida do chamado.

Atributo	Tipo	Restrições
id_status (PK)	SERIAL	NOT NULL
descricao	VARCHAR	NOT NULL (Aberto, Em Atendimento, Resolvido, Cancelado)

categorias — classificação dos chamados.

Atributo	Tipo	Restrições
id_categoria (PK)	SERIAL	NOT NULL
nome	VARCHAR	NOT NULL (Hardware, Software, Rede, Acesso)
descricao	VARCHAR	NULL

chamados — entidade central.

Atributo	Tipo	Restrições
id_chamado (PK)	SERIAL	NOT NULL
titulo	VARCHAR	NOT NULL
descricao	TEXT	NOT NULL
prioridade	VARCHAR	NOT NULL (Baixa / Média / Alta)
data_abertura	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
data_fechamento	TIMESTAMP	NULL (preenchida automaticamente ao Resolver/Cancelar)
id_status (FK)	INT	NOT NULL → status.id_status
id_categoria (FK)	INT	NOT NULL → categorias.id_categoria
id_tecnico (FK)	INT	NULL → tecnicos.id_tecnico
id_departamento (FK)	INT	→ departamentos.id_departamento
id_usuario (FK)	INT	NOT NULL → usuarios.id_usuario

historico_chamados — linha do tempo de movimentações.

Atributo	Tipo	Restrições
id_historico (PK)	SERIAL	NOT NULL
id_chamado (FK)	INT	NOT NULL → chamados.id_chamado
status_anterior	VARCHAR	NULL
status_novo	VARCHAR	NOT NULL
id_tecnico (FK)	INT	NULL → tecnicos.id_tecnico
data_hora	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
descricao	TEXT	NULL

4.2 Relacionamentos (cardinalidade 1:N)

Relacionamento	Cardinalidade	Descrição
usuarios → chamados	1:N	Um usuário (solicitante) abre vários chamados; cada chamado tem um solicitante.
departamentos → chamados	1:N	Um departamento origina vários chamados; cada chamado pertence a um departamento.
departamentos → tecnicos	1:N	Um departamento possui vários técnicos.
categorias → chamados	1:N	Uma categoria classifica vários chamados.
status → chamados	1:N	Um status vale para vários chamados.
tecnicos → chamados	1:N	Um técnico atende vários chamados.
chamados → historico_chamados	1:N	Um chamado acumula várias movimentações em seu histórico.

5. Arquitetura MVC e Organização de Pastas

O sistema segue o padrão **MVC (Model-View-Controller)**: os *models* encapsulam o acesso ao PostgreSQL (driver `pg`), os *controllers* concentram a lógica de aplicação e as regras de negócio, as *routes* mapeiam as URLs para os controllers, as *views* (EJS) renderizam a interface com Bootstrap 5 e os *middlewares* tratam autenticação e proteção de rotas.

Estrutura **real** do repositório (branch `main`) no momento da consolidação:

```

HelpDeskTi_TFinal/
├─ app.js                # ponto de entrada (Express, sessão, rotas, EJS)
├─ package.json          # dependências e scripts (dev, prod, db:setup)
├─ package-lock.json
├─ .env.example          # modelo das variáveis de ambiente
├─ README.md
├─ LICENSE
├─ public/              # arquivos estáticos (css, js, imagens)
├─ database/
|   └─ db.js            # pool de conexão PostgreSQL
|   └─ schema.sql       # criação das 7 tabelas
|   └─ seed.sql         # dados iniciais de demonstração
|   └─ setup.js         # script de criação/popular banco (db:setup)
└─ src/
    ├─ controllers/
    |   └─ authController.js
    |   └─ chamadoController.js
    |   └─ departamentoController.js
    ├─ middlewares/
    |   └─ authMiddleware.js    # proteção de rotas / controle de acesso
    ├─ models/
    |   └─ Usuario.js
    |   └─ Chamado.js
    |   └─ Departamento.js
    ├─ routes/
    |   └─ authRoutes.js
    |   └─ chamadoRoutes.js
    |   └─ departamentoRoutes.js
    └─ views/
        ├─ partials/          # layout-start, layout-end, sidebar
        ├─ auth/              # login.ejs
        ├─ chamados/          # index, novo, editar, historico
        ├─ departamentos/     # index, novo, editar
        ├─ dashboard.ejs
        └─ index.ejs

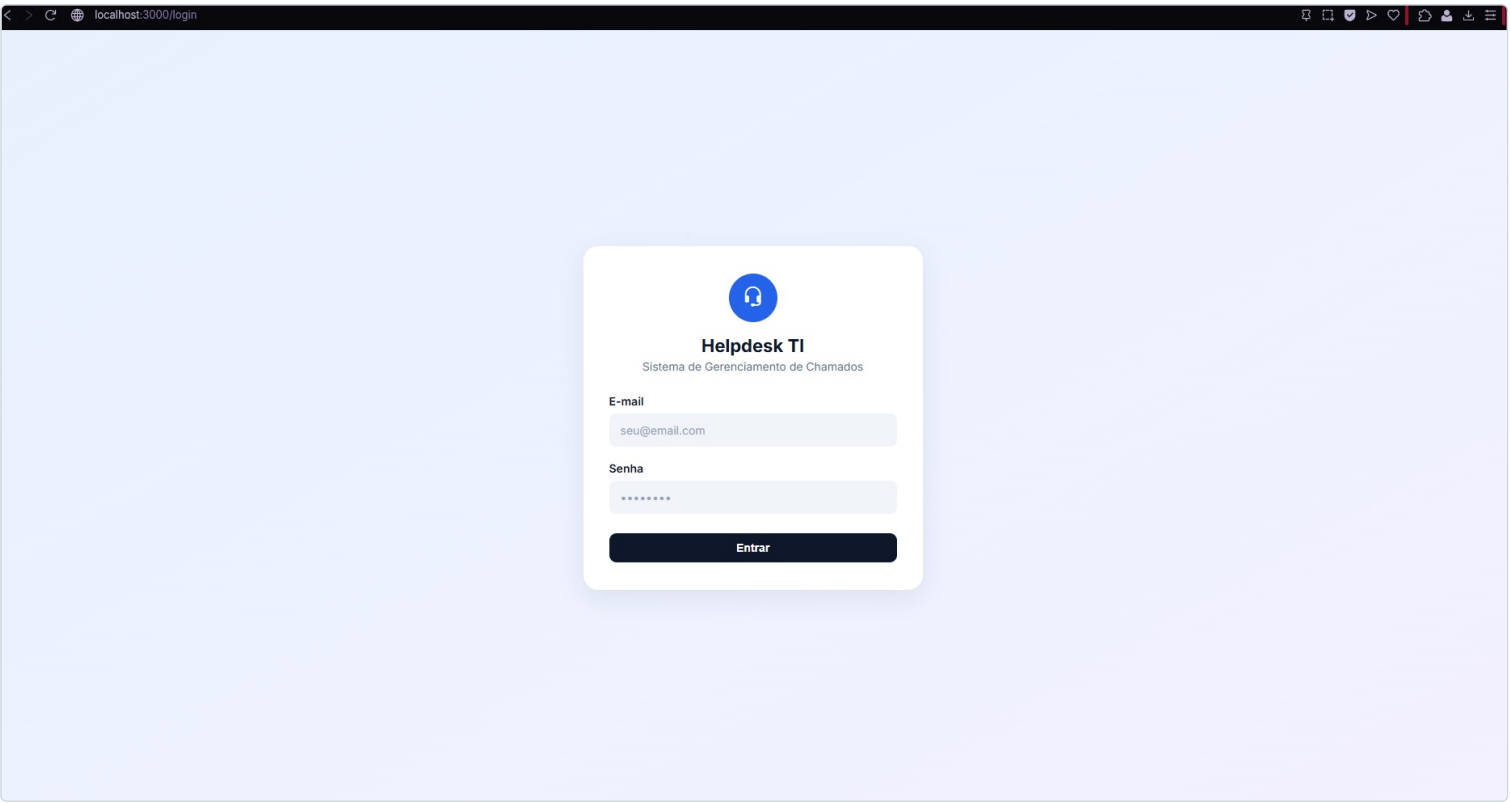
```

Os módulos de **Técnicos** e **Categorias** (controllers, models, routes e views próprios) estão **em desenvolvimento** em *feature branches* e serão integrados ao `main` antes da defesa. O fluxo de trabalho adota *feature branches* (ex.: `feat-CRUD-chamados`, `feat-CRUD-departamentos-e-autenticacao`) com integração à branch principal.

6. Telas do Sistema

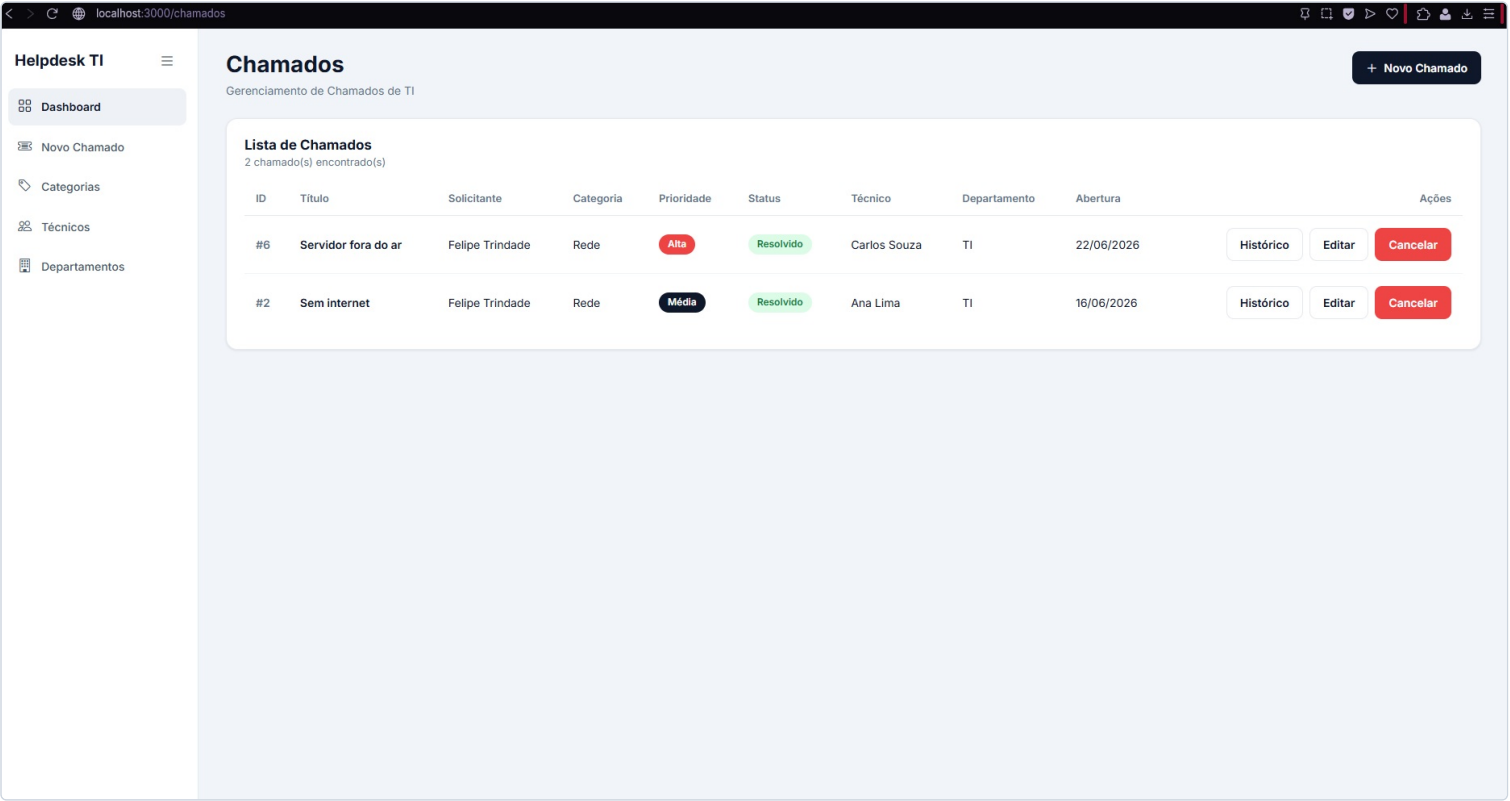
As figuras a seguir apresentam as telas implementadas. A navegação lateral (partial `sidebar`) reúne os atalhos dos módulos do sistema — Dashboard, Novo Chamado, Categorias, Técnicos e Departamentos —, sendo que **Categorias** e **Técnicos** correspondem a módulos em desenvolvimento. Observa-se que a *prioridade* não constitui um CRUD separado: é um campo do próprio chamado (Baixa/Média/Alta), definido no momento da abertura.

Figura 1 — Tela de Login (`/login`)



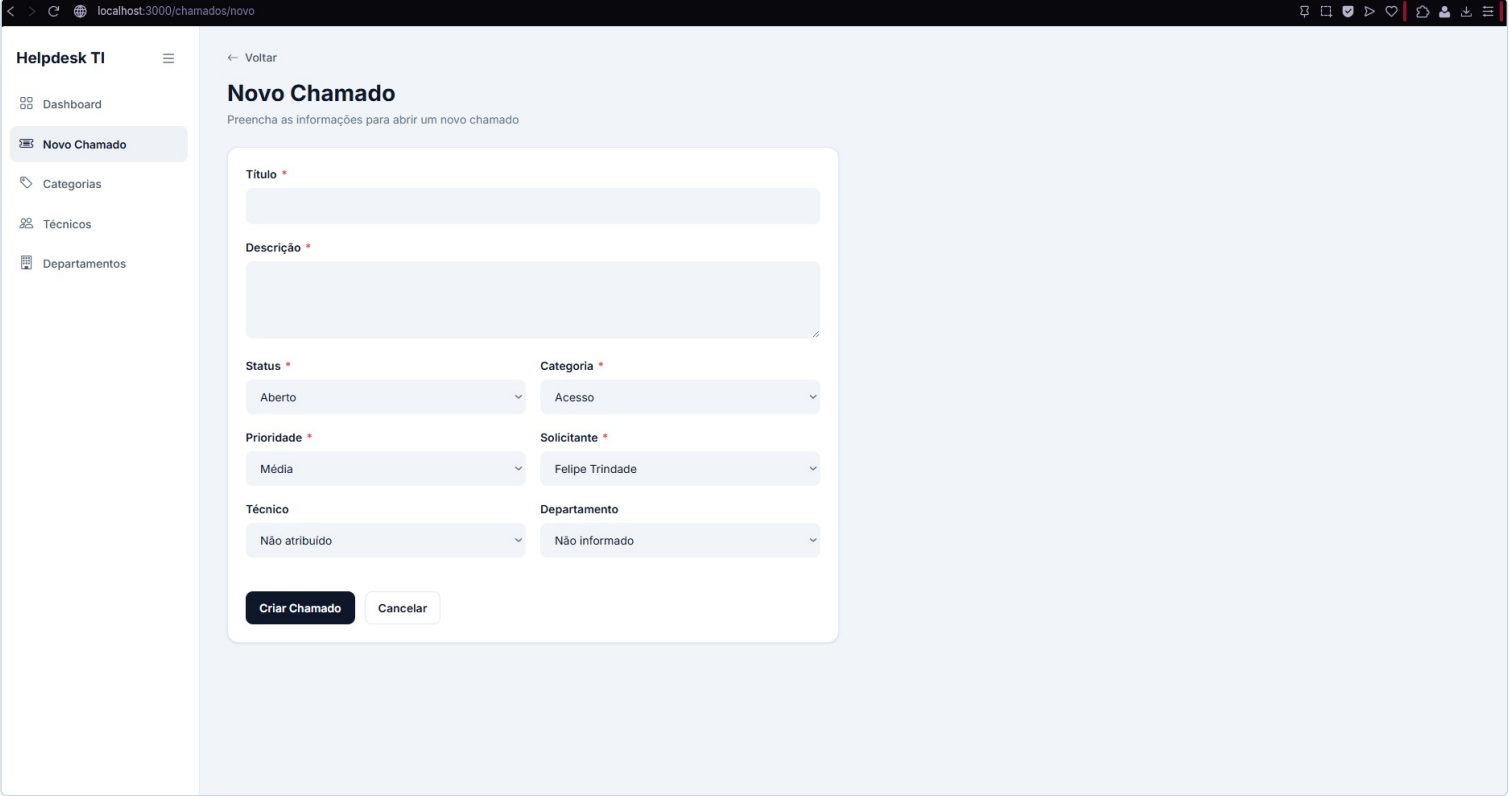
Página de autenticação "Helpdesk TI — Sistema de Gerenciamento de Chamados", com campos de **E-mail** e **Senha** e botão **Entrar**.

Figura 2 — Lista de Chamados (/chamados)



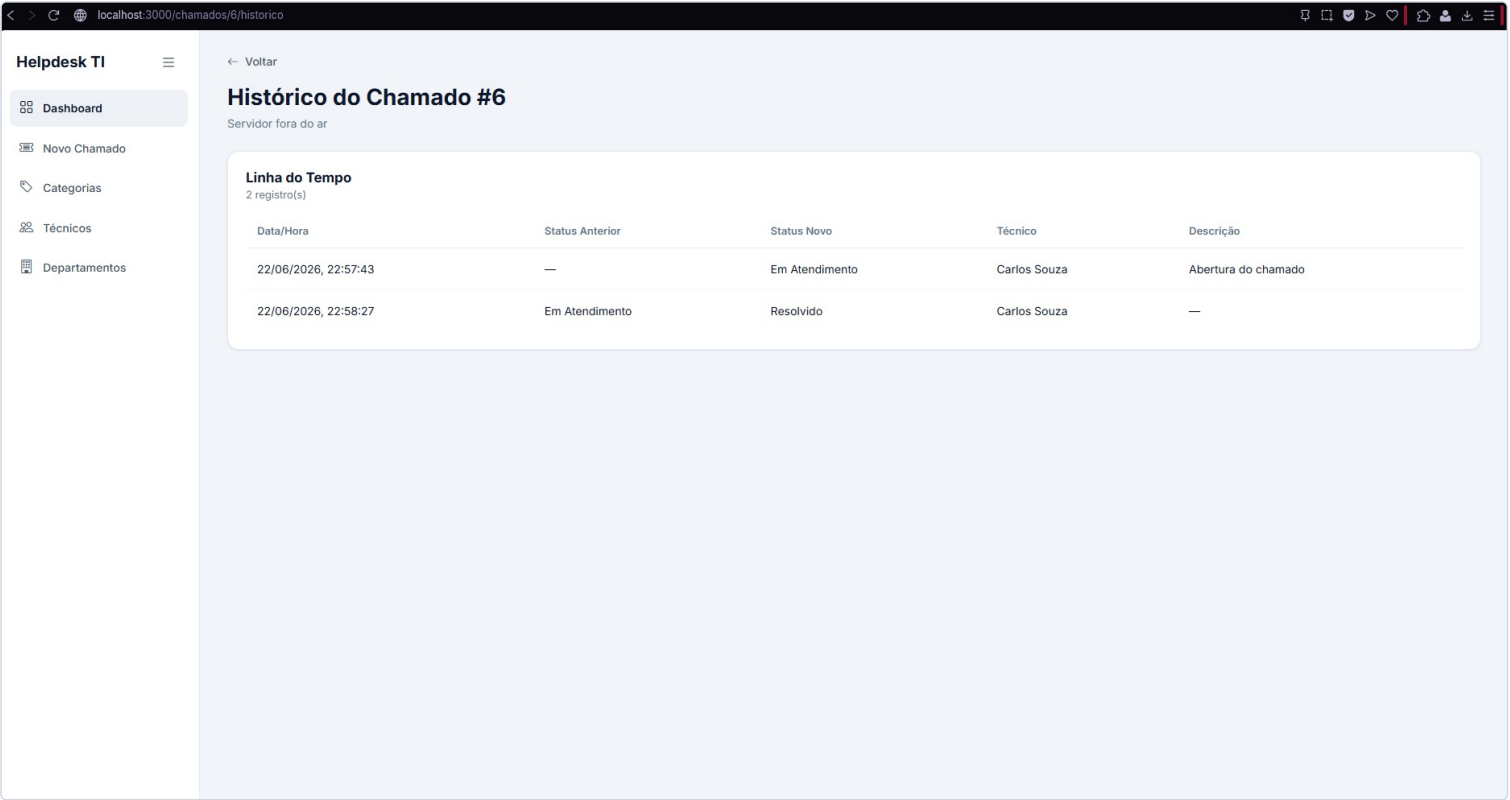
Tela principal "Gerenciamento de Chamados de TI", exibindo a lista com colunas ID, Título, Solicitante, Categoria, Prioridade, Status, Técnico, Departamento e Abertura. Cada linha oferece as ações **Histórico**, **Editar** e **Cancelar**, além do botão **Novo Chamado**. Os *badges* de prioridade (ex.: Alta, Média) e de status (ex.: Resolvido) facilitam a leitura visual.

Figura 3 — Novo Chamado (/chamados/novo)



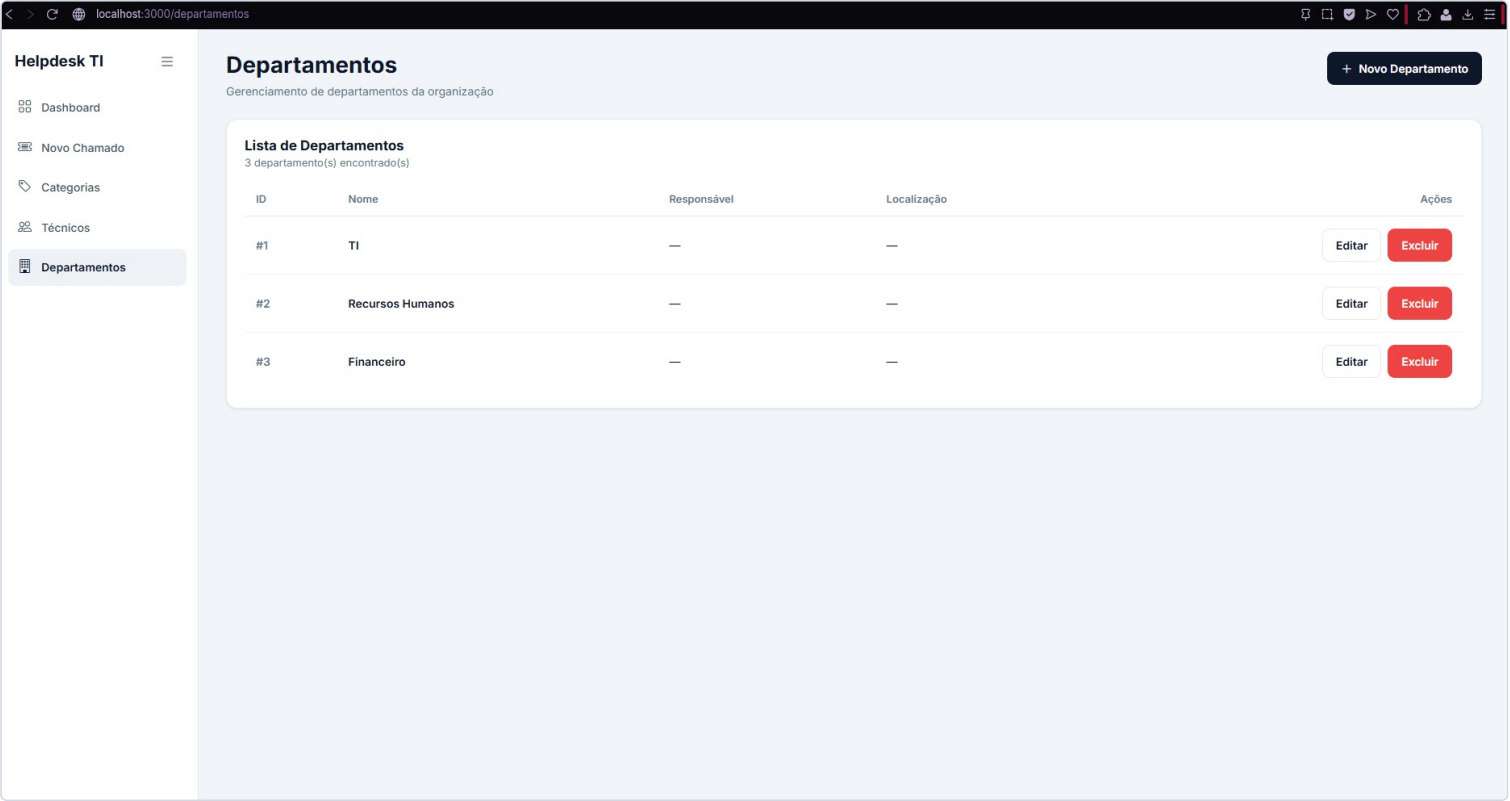
Formulário de abertura com **Título** e **Descrição** obrigatórios, e seletores de **Status**, **Categoria**, **Prioridade** (Baixa/Média/Alta), **Solicitante**, **Técnico** e **Departamento**. Ações **Criar Chamado** e **Cancelar**.

Figura 4 — Histórico do Chamado (/chamados/:id/historico)



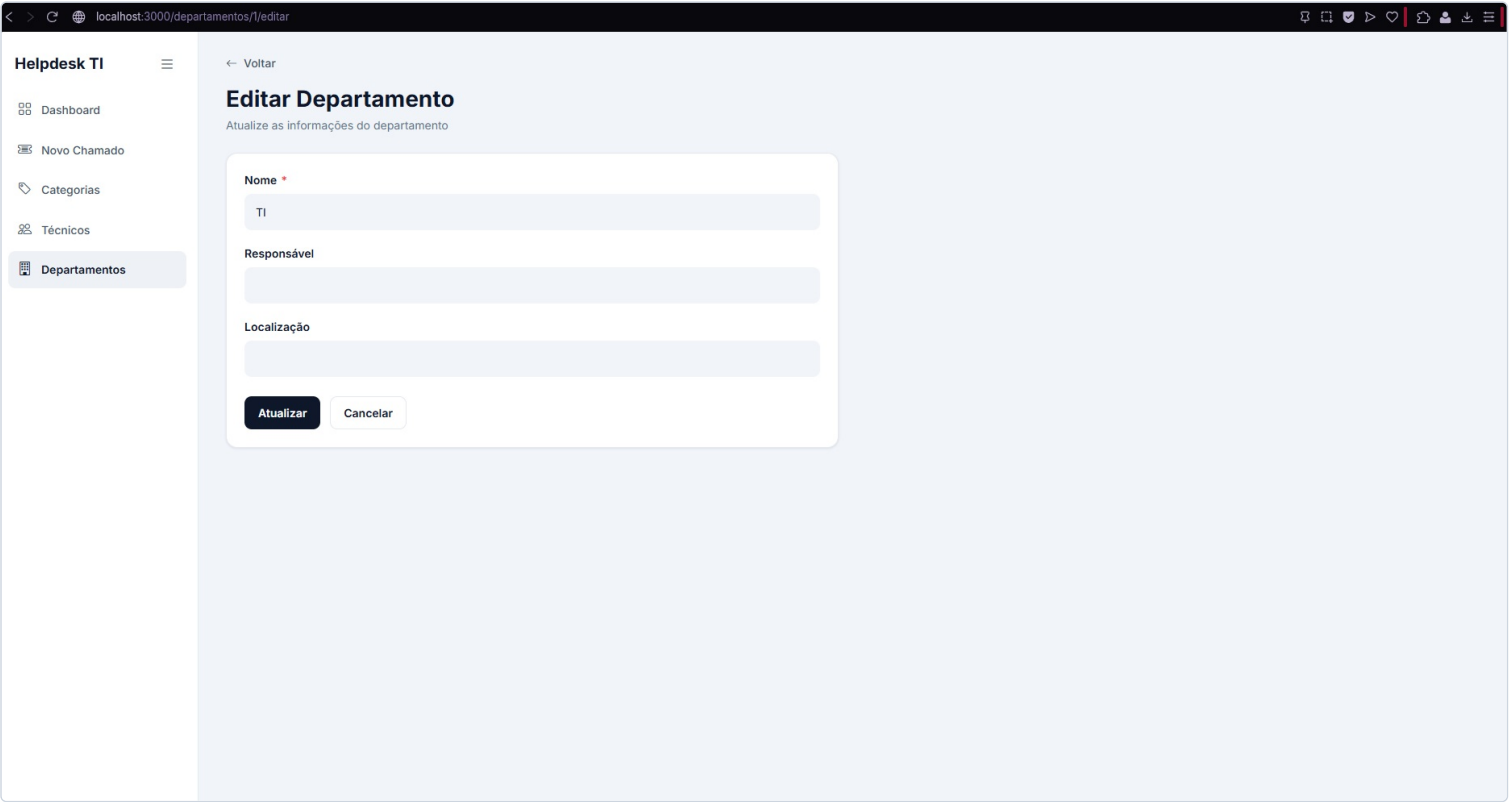
Linha do tempo do chamado, com Data/Hora, Status Anterior, Status Novo, Técnico e Descrição. Cada movimentação (abertura e mudança de status) gera um registro consultável — evidência do histórico automático.

Figura 5 — Departamentos (/departamentos)



CRUD de departamentos, com colunas ID, Nome, Responsável, Localização e ações **Editar/Excluir**, além do botão **Novo Departamento**.

Figura 6 — Editar Departamento (/departamentos/:id/editar)



Formulário de edição de departamento, com o campo **Nome** (obrigatório) e os campos **Responsável** e **Localização**, e as ações **Atualizar** e **Cancelar**.

7. Funcionalidades por Módulo

Módulo / Funcionalidade	Detalhamento	Responsável	Status
Autenticação	Login, logout, sessão (<code>express-session</code>), proteção de rotas por middleware, hash de senha com bcrypt , geração de token JWT no login e controle de acesso por perfil (admin x comum).	Higor Bueno	Finalizado
CRUD de Chamados	Abertura, edição e listagem de chamados com prioridade (Baixa/Média/Alta); data de fechamento automática ao resolver/cancelar; cancelamento como <i>soft delete</i> (preserva o histórico); histórico automático — toda abertura e mudança de status gera registro consultável em <code>/chamados/:id/historico</code> .	Felipe Trindade	Finalizado
CRUD de Departamentos	Cadastro, edição, listagem e exclusão de departamentos.	Higor Bueno	Finalizado
CRUD de Técnicos	Cadastro, edição e listagem de técnicos, com vínculo a departamento.	Fernando Vinicius	Em desenvolvimento
CRUD de Categorias	Cadastro, edição e listagem de categorias de chamados.	Nicolas Toshio	Em desenvolvimento
Dashboard com indicadores	Painel inicial com contadores e visão geral das movimentações.	Fernando Vinicius	Em desenvolvimento

8. Instruções de Execução

Pré-requisitos: Node.js (20.x), npm e PostgreSQL instalados e em execução.

- Clonar o repositório:** `bash git clone https://github.com/Trindadefelipe/HelpDeskTi_TFinal.git cd HelpDeskTi_TFinal`
- Instalar as dependências:** `bash npm install`
- Configurar variáveis de ambiente:** copie `.env.example` para `.env` e preencha as credenciais do PostgreSQL, além de `SESSION_SECRET` e `JWT_SECRET`: `bash cp .env.example .env`
- Criar o banco, as tabelas e os dados iniciais:** `bash npm run db:setup`
- Iniciar a aplicação** (modo desenvolvimento, com recarga automática): `bash npm run dev` (para execução padrão, use `npm run prod.`)
- Acessar:** `http://localhost:3000`

Logins de teste (criados pelo seed):

Perfil	E-mail	Senha
Administrador	felipe@empresa.com	123456
Comum	maria@empresa.com	123456

9. Testes Manuais (Postman)

Os testes foram realizados de forma **manual via Postman**, organizados por CRUD na pasta `docs/TestePostman/` do repositório. Cada integrante mantém sua própria subpasta com as evidências das requisições do módulo sob sua responsabilidade (requisições de criação, leitura, atualização e remoção/cancelamento, bem como os fluxos de autenticação e de histórico).

A abordagem de teste cobriu, para cada endpoint:

- Validação de campos obrigatórios e respostas de erro amigáveis (sem exposição de *stack trace*).
- Verificação das regras de negócio (preenchimento automático de data de abertura, fechamento ao resolver/cancelar e geração de registro de histórico a cada mudança de status).
- Conferência da proteção de rotas (acesso autenticado x não autenticado) e do controle de acesso por perfil.

Observação: confirmar antes da defesa que a pasta `docs/TestePostman/` (com as evidências por integrante) está commitada na branch entregue.

10. Uso de Inteligência Artificial

A IA foi utilizada **exclusivamente como ferramenta de apoio**, sempre sob revisão humana antes da incorporação ao projeto. A lógica dos CRUDs e o código de produção são de autoria da equipe. O detalhamento completo (ferramentas, prompts, decisões aceitas/adaptadas/recusadas e reflexão crítica) é mantido no arquivo `USO_IA.md` do repositório.

Principais frentes de apoio da IA:

- Explicação de conceitos:** arquitetura MVC, middlewares no Express, hash com bcrypt, controle de sessão, JWT e versionamento por *feature branches*.

- **Padrão de layout:** geração do padrão visual das telas a partir dos protótipos elaborados no Figma.
- **Revisão de código:** leitura crítica de trechos e sugestões de melhoria, com reimplementação pela equipe.
- **Explicação de erros:** apoio na interpretação de mensagens de erro durante o desenvolvimento.
- **Documentação e testes:** estruturação dos documentos de entrega e apoio à organização dos testes.

A decisão da equipe de não delegar à IA a escrita do código de produção fortalece o aprendizado prático e sustenta a defesa individual de cada integrante sobre o módulo que declarou como seu.

11. Matriz de Papéis e Responsabilidades da Equipe

Integrante	Módulo principal	Funções transversais	Status do módulo
Felipe Trindade	Chamados (entidade central + histórico automático + <i>soft delete</i>)	Setup do projeto (Node/Express/EJS/PostgreSQL), consolidação da documentação	Finalizado
Higor Bueno	Departamentos + Autenticação	Autenticação (bcrypt + sessão + JWT), middlewares de acesso, README e US0_IA.md	Finalizado
Fernando Vinicius	Técnicos + Dashboard	Layout base, Bootstrap 5 e protótipos de tela	Em desenvolvimento
Nicolas Toshio	Categorias	Modelagem do banco (<code>schema.sql</code> / <code>seed.sql</code>) e DER	Em desenvolvimento
IA (Assistente)	— (não desenvolve código)	Apoio: explicação de conceitos, padrão de layout, revisão de código, explicação de erros, documentação e testes	—
Todos	—	Revisão de commits, atualização do Kanban no GitHub Projects, manutenção do US0_IA.md	—

12. Considerações Finais

A Entrega Final consolida o Sistema Helpdesk de TI como aplicação web fullstack executável, em arquitetura MVC, com banco PostgreSQL real, autenticação de usuários (sessão, bcrypt e JWT), controle de acesso por perfil e os módulos centrais já operacionais: **Autenticação**, **CRUD de Chamados** (com histórico automático e cancelamento por *soft delete*) e **CRUD de Departamentos**.

Os módulos de **Técnicos**, **Categorias** e o **Dashboard de indicadores** encontram-se **em desenvolvimento**, em *feature branches*, com integração à branch principal prevista para a reta final até a defesa (01/07/2026). A base arquitetural já estabelecida — separação por camadas, *seed* de demonstração, *middlewares* de proteção e histórico automático — sustenta a conclusão desses módulos seguindo o mesmo padrão dos CRUDs já entregues.

O documento `US0_IA.md`, mantido no repositório, registra de forma transparente o uso responsável das ferramentas de IA como apoio, preservando a autoria do código e o aprendizado prático da equipe.